

Automated CI/CD and Real-Time Failure Monitoring of a Containerized Environmental Sensor Dashboard Using Jenkins, Docker, Prometheus and Alertmanager

DevOps Lab CA – Mini Project Report

Yashasvi Gupta (Roll no. _____)

_____ (Roll no. _____)

_____ (Roll no. _____)

Batch _____ · Guide: _____ · October 2026

1. Abstract
2. Introduction
 - 2.1 Background
 - 2.2 Problem statement
 - 2.3 Objectives
3. Tools and technologies
4. System architecture
5. Implementation
 - 5.1 The application
 - 5.2 Version control workflow
 - 5.3 Containerization
 - 5.4 CI/CD pipeline (Jenkins)
 - 5.5 Monitoring with Prometheus
 - 5.6 Alerting with Alertmanager
6. Testing
7. Results
 - 7.1 Pipeline runs
 - 7.2 Failure detection
8. Challenges and how we solved them
9. Conclusion
10. Future scope
11. References

1. Abstract

We built a small web application, an environmental sensor dashboard, and then automated everything that happens to it after a developer commits code. Every push to GitHub is picked up by Jenkins, which runs the unit tests inside a Docker container, validates the monitoring configuration, builds a new image, deploys it with Docker Compose and smoke-tests the result. If the tests fail, nothing is deployed; if the smoke test fails, the previous working image is restored automatically. The running system is watched by Prometheus, which scrapes the application every five seconds and evaluates seven alert rules. Alertmanager groups, routes and de-duplicates the resulting alerts and delivers them to a webhook receiver that stands in for Slack or email. Failed pipeline runs are reported through the same channel. We tested the whole loop with real failure scenarios: overheating and offline sensors, a pollution spike, an API returning errors, a crashed process, a stopped container, a commit with a broken test and a release that fails its health check. In every case the problem was either blocked before deployment or detected and reported within about 30 to 50 seconds, and each alert was marked resolved once the problem was fixed.

Source code: https://github.com/Yashvi2874/DevOps_failure_monitoring

2. Introduction

2.1 Background

A DevOps workflow closes the gap between writing code and running it. Instead of a person copying files to a server and hoping nothing breaks, the steps from commit to production are written down as code and run by machines, and the running system is measured continuously so that problems are found by monitoring rather than by users.

Our application is deliberately simple: a dashboard that shows temperature, humidity, CO₂ and PM2.5 readings from three sensors placed in a lab, a server room and outdoors. The readings are simulated, because the subject of the project is the delivery pipeline and the monitoring around the app, not the sensor hardware. A simple app also makes it easy to break things on purpose and watch how the system reacts.

2.2 Problem statement

Without automation, a team faces three recurring problems:

1. **Untested code reaches users.** Someone forgets to run the tests before deploying, and a bug goes live.
2. **"It works on my machine."** The app behaves differently on the developer's laptop, the tester's machine and the server, because each has different library versions.
3. **Failures are noticed late.** A sensor stops reporting or the server room overheats, and nobody finds out until someone happens to look.

2.3 Objectives

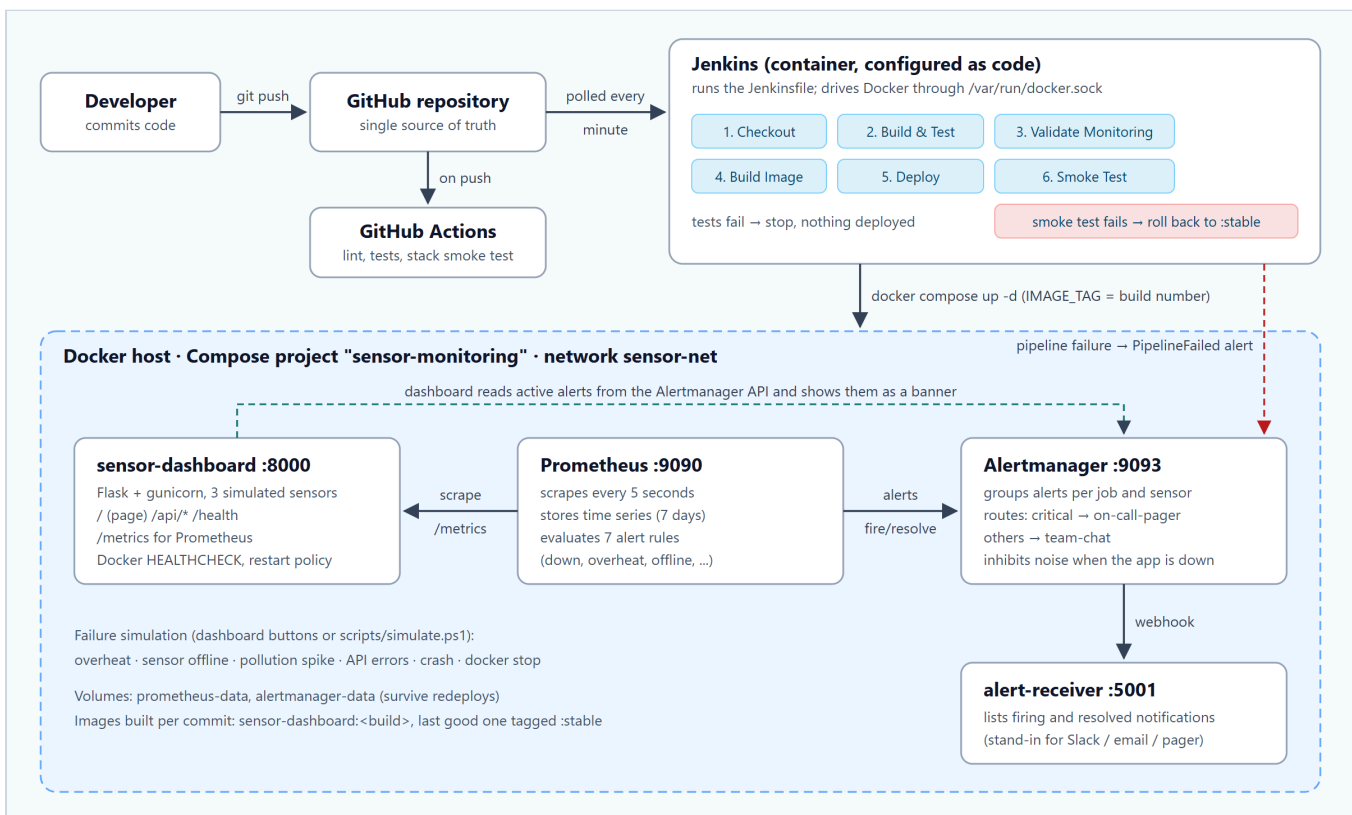
1. Keep all code and configuration in Git, using feature branches.
2. Build the application into a Docker image so it runs identically everywhere.
3. Run automated tests on every change and stop the pipeline if any fail.

4. Deploy automatically with Jenkins, and roll back automatically if the new version is unhealthy.
5. Collect metrics from the running app with Prometheus.
6. Define alert rules for application and sensor failures, and deliver alerts through Alertmanager with sensible grouping, routing and noise suppression.
7. Demonstrate each failure scenario and measure how long detection takes.

3. Tools and technologies

Area	Tool (version)	Why we used it
Version control	Git, GitHub	History of every change; Jenkins and GitHub Actions read the code from here
Application	Python 3.12, Flask 3.1, gunicorn 23	Small, readable web stack
Metrics library	prometheus_client 0.23	Exposes app and sensor metrics in Prometheus format
Testing	pytest 8.4, pytest-cov, flake8 7.3, promtool, unittest	Unit tests, coverage, lint, and tests for the alert rules themselves
Containers	Docker Engine 29, Docker Compose plugin	Packaging and running all five services
CI/CD	Jenkins 2.580.1 LTS (declarative pipeline, Configuration as Code, Job DSL)	Automates test → build → deploy → verify
CI on GitHub	GitHub Actions	A second, independent check on every push
Monitoring	Prometheus 3.13.4	Pull-based metrics collection and alert rule evaluation
Alerting	Alertmanager 0.34.1	Grouping, routing, inhibition and delivery of alerts

4. System architecture



Overall architecture of the project

The system has two halves.

The delivery half starts with the developer. Code is pushed to GitHub, where GitHub Actions runs a quick check and Jenkins picks the change up by polling the repository every minute. Jenkins itself runs in a Docker container. It has the Docker command-line tools installed and the host's Docker socket mounted, so every pipeline step (building images, running tests, starting containers) is carried out by the host's Docker engine.

The running half is a Docker Compose project called `sensor-monitoring` with four containers on a private network named `sensor-net`:

Container	Port	Role
sensor-dashboard	8000	The Flask app: dashboard page, JSON API, /health, /metrics
prometheus	9090	Scrapes /metrics every 5 s, stores the time series, evaluates alert rules
alertmanager	9093	Receives alerts from Prometheus (and Jenkins), groups and routes them
alert-receiver	5001	Shows every notification Alertmanager delivers

Containers find each other by service name: Prometheus scrapes `dashboard:8000`, and the dashboard reads active alerts from `alertmanager:9093` so it can show them as a banner on the page.

5. Implementation

5.1 The application

The sensor simulator (`app/sensor_dashboard/sensors.py`) keeps three sensors, each with its own baseline. On every tick (every two seconds) each value moves a little towards its baseline plus some random noise, which gives realistic, slowly drifting readings. Each sensor can be switched into a failure scenario:

- **overheat**: the temperature drifts towards 45 °C;
- **offline**: the sensor stops producing readings and every attempted read counts as an error;
- **pollution**: CO₂ and PM2.5 climb towards unhealthy levels.

The Flask app (`app.py`) serves the dashboard page, which refreshes itself every two seconds, and a small JSON API. Two more endpoints matter for operations:

- `/health` returns the app's status and the build number that is running. The Jenkins smoke test uses the build number to make sure the *new* version is the one answering.
- `/metrics` exposes the readings and the app's own behaviour in Prometheus format:

Metric	Meaning
<code>sensor_temperature_celsius</code> , <code>sensor_humidity_percent</code> , <code>sensor_co2_ppm</code> , <code>sensor_pm25_ugm3</code>	Latest reading per sensor
<code>sensor_last_seen_timestamp_seconds</code>	When each sensor last reported (used to detect offline sensors)
<code>sensor_up</code> , <code>sensor_read_errors_total</code>	Sensor status and failed reads
<code>dashboard_http_requests_total</code> , <code>dashboard_http_request_duration_seconds</code>	Request count and latency per endpoint and status code
<code>dashboard_build_info</code>	Which build and commit are running
<code>process_start_time_seconds</code>	When the process started (used to detect restarts)

A **failure simulation** panel on the dashboard (and the scripts `scripts/simulate.ps1` and `simulate.sh`) can trigger each scenario, make the API return HTTP 500 errors, or crash the app.

5.2 Version control workflow

Each feature was developed on its own branch and merged into `main` with a merge commit, so the history shows where each feature began and ended: `feature/sensor-simulator`, `feature/dashboard`, `feature/docker`, `feature/monitoring`, `feature/jenkins-pipeline`, `feature/github-actions`, and a later `fix/pipeline-alerts`. Secrets such as the Jenkins admin password live in `jenkins/.env`, which is listed in `.gitignore` and never committed.

Platform Solutions Resources Open Source Enterprise Pricing

Search / Sign in Sign up

Yashvi2874 / DevOps_failure_monitoring Public

Notifications Fork 0 Star 0

Code Issues Pull requests Actions Projects Security and quality Insights

Commits

main

Commits on Oct 2, 2026

Revert the deliberately failing test	aef0aea
Demo: push a failing test to show the pipeline blocks it	228122f
Merge branch 'fix/pipeline-alerts'	0fa0ee6
Clearer pipeline alerts, grouped separately from app alerts	83bc819
Merge branch 'feature/github-actions'	d4e9057
Merge branch 'feature/jenkins-pipeline'	eed2690
Check every push on GitHub Actions too	be04f05
Run Jenkins in Docker, configured as code	f531335

Commit history on GitHub

5.3 Containerization

app/Dockerfile is a multi-stage build:

```
FROM python:3.12-slim AS base      # Python + runtime libraries
...
FROM base AS test                  # + pytest and flake8; runs the tests when started
...
FROM base AS runtime               # the image that gets deployed
RUN useradd --create-home --uid 10001 appuser
USER appuser
HEALTHCHECK --interval=10s CMD python -c "import urllib.request;
    urllib.request.urlopen('http://127.0.0.1:8000/health')"
CMD ["gunicorn", "--bind", "0.0.0.0:8000", "--workers", "1", "--threads", "4",
    "sensor_dashboard.wsgi:app"]
```

The `requirements.txt` file is copied and installed before the source code, so Docker can reuse the cached dependency layer when only the code changes. The runtime image runs as a non-root user, and has a health check that Docker uses to mark the container healthy or unhealthy. The Compose file sets `restart: unless-stopped`, so a crashed container is started again automatically.

The Prometheus and Alertmanager configuration files are copied into small custom images rather than mounted from the host. This was a deliberate choice (see section 8): it lets Jenkins deploy the stack even though Jenkins runs in its own container, and it means a change to an alert rule is built, tested and deployed exactly like a change to the code.

5.4 CI/CD pipeline (Jenkins)

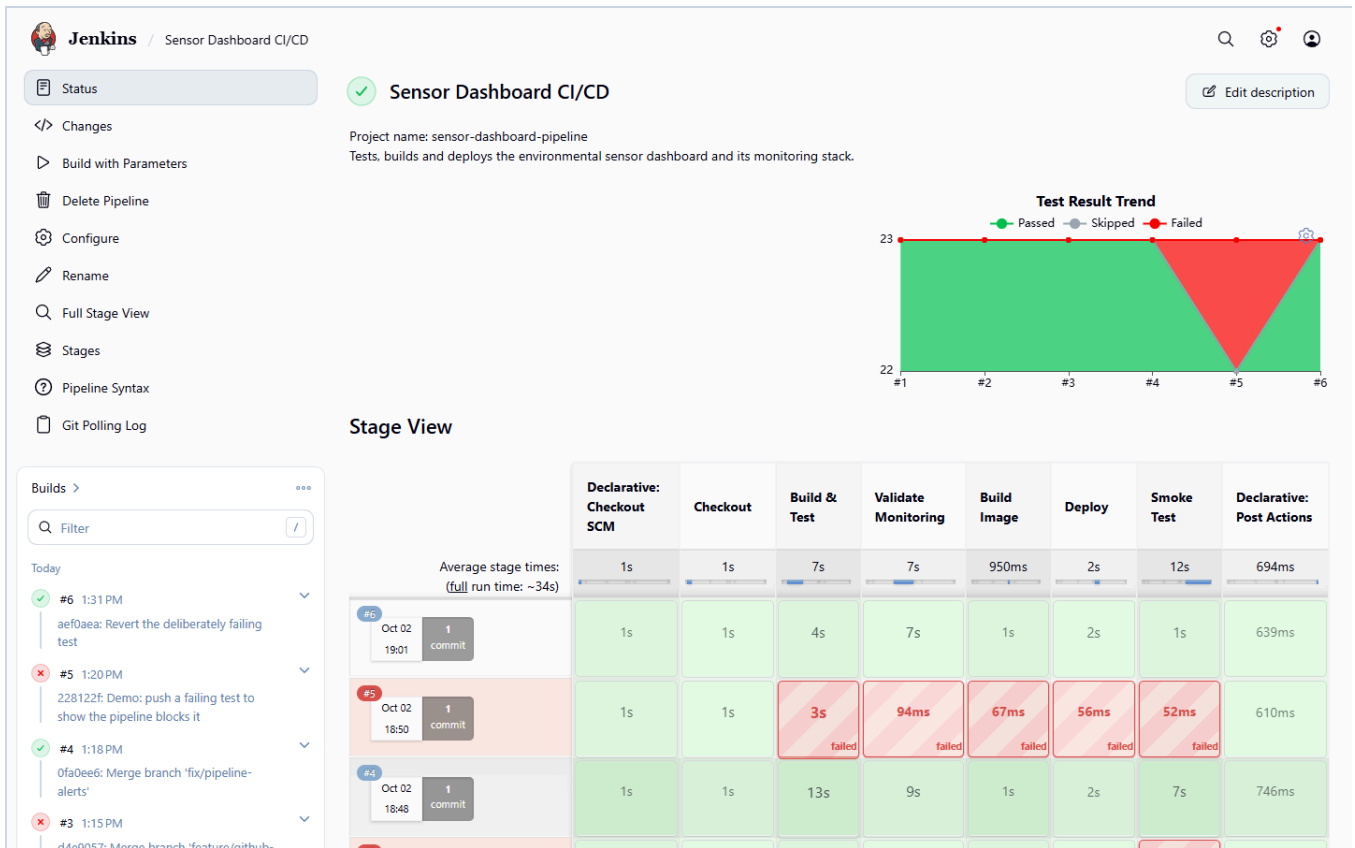
The pipeline is defined in the `Jenkinsfile` at the root of the repository:

Stage	What it does	On failure
Checkout	Fetches the commit and records its hash and message	—
Build & Test	Builds the <code>test</code> image and runs <code>flake8</code> and 23 <code>pytest</code> tests inside a container; publishes the JUnit report	Pipeline stops, nothing is deployed
Validate Monitoring	Checks the Prometheus config, runs the alert-rule unit tests, checks the Alertmanager config and its routing, runs the receiver's tests	Pipeline stops
Build Image	Builds <code>sensor-dashboard:<build number></code> with the version, commit hash and build number baked in	Pipeline stops
Deploy	<code>docker compose up -d</code> with <code>IMAGE_TAG=<build number></code> ; only containers whose image changed are replaced	Pipeline stops
Smoke Test	From a throwaway container on <code>sensor-net</code> : <code>/health</code> is OK and reports this build, the API returns three sensors, <code>/metrics</code> works, and Prometheus is scraping the new container	<code>rollback.sh</code> redeploys <code>sensor-dashboard:stable</code> and checks it

When the smoke test passes, the image is also tagged `sensor-dashboard:stable`, which marks it as the last known good build for future rollbacks. At the end, the pipeline sends its result to Alertmanager: a failed build raises a `PipelineFailed` alert and the next successful build resolves it. This way, broken deployments and broken builds reach the team through the same channel as application failures.

Jenkins is set up entirely from files in `jenkins/`. The `Dockerfile` adds the Docker CLI, the Compose plugin and the required Jenkins plugins. `casc.yaml` (Jenkins Configuration as Code) creates the admin user and, through the Job DSL plugin, the `sensor-dashboard-pipeline` job, which polls GitHub every minute. There is no setup wizard. The whole CI server can be rebuilt with one `docker compose up` command.

GitHub Actions (`.github/workflows/ci.yml`) runs lint, the unit tests, the rule tests and a full stack smoke test on GitHub's own machines for every push, as an independent second check.



Jenkins stage view: builds #1–#6

5.5 Monitoring with Prometheus

Prometheus scrapes three targets every five seconds: the dashboard, itself and Alertmanager. It evaluates the rules in `alert_rules.yml` at the same interval. Each rule is a PromQL query plus a `for` duration, which means the condition has to hold for that long before the alert fires. This avoids alerts for momentary blips.

Alert	Condition	Severity
SensorDashboardDown	<code>up == 0 for 30 s</code>	critical
HighApiErrorRate	more than 20% of <code>/api/*</code> requests are 5xx, for 30 s	critical
SensorDashboardRestarted	<code>process_start_time_seconds</code> changed in the last 5 min	info
SensorOverheat	temperature > 35 °C for 15 s	warning
SensorOffline	no reading for more than 10 s, for 10 s	warning
HighCO2Level	CO ₂ > 1500 ppm for 20 s	warning
HighPM25Level	PM2.5 > 100 µg/m ³ for 20 s	warning

The rules have their own unit tests (`monitoring/prometheus/tests/alert_rules_test.yml`), run with `promtool test rules`. Each test feeds Prometheus a made-up time series and checks which alerts are firing at a given moment, including the exact alert text. For example, one test checks that a 10-second outage does *not* page anyone. We confirmed the tests are real by changing one expected value and watching the test fail.

Prometheus

Query

Alerts

Status > Target health

Select scrape pool

Filter by target health

Filter by endpoint or labels

alertmanager

1 / 1 up

Endpoint

Labels

Last scrape

State

<http://alertmanager:9093/metrics>

instance="alertmanager:9093"

job="alertmanager"

4.966s ago

10ms

UP

prometheus

1 / 1 up

Endpoint

Labels

Last scrape

State

<http://localhost:9090/metrics>

instance="localhost:9090"

job="prometheus"

1.185s ago

36ms

UP

sensor-dashboard

1 / 1 up

Endpoint

Labels

Last scrape

State

<http://dashboard:8000/metrics>

app="sensor-dashboard"

instance="dashboard:8000"

job="sensor-dashboard"

3.289s ago

6ms

UP

Prometheus target health: all three targets up

5.6 Alerting with Alertmanager

Alertmanager decides who hears about an alert, when, and how often:

- **Routing.** Critical alerts go to an `on-call-pager` receiver and are sent after 5 seconds; everything else goes to `team-chat`.
- **Grouping.** Alerts are grouped by job and sensor. A pollution spike on Sensor 3 raises both `HighCO2Level` and `HighPM25Level`, but they arrive as a single notification.
- **Inhibition.** While `SensorDashboardDown` is firing, the app's warning and info alerts are held back, because they are symptoms of the same outage. An offline sensor's last temperature is stale, so `SensorOffline` also suppresses `SensorOverheat` for that sensor.
- **Resolution.** `send_resolved: true` means a second notification is sent when a problem clears.

Both receivers are webhooks pointing at our `alert-receiver` service, a small Python program (standard library only) that stores the notifications and lists them on a web page. In a real team this would be Slack, email or a paging service. Only the receiver section of `alertmanager.yml` would change.

6. Testing

Level	What is tested	Where it runs
Lint	Code style and obvious mistakes (flake8)	Jenkins, GitHub Actions
Unit tests (23)	Sensor simulation, failure scenarios, every API endpoint, metrics output, request counting, simulation switches, alert formatting. Coverage is 91%	Jenkins (inside Docker), GitHub Actions
Alert rule tests (6 scenarios)	Each alert fires at the right time with the right labels and text, and short blips don't fire	Jenkins, GitHub Actions
Config validation	Prometheus config, Alertmanager config, routing of critical vs warning alerts	Jenkins, GitHub Actions
Receiver tests (5)	Parsing, formatting, HTML escaping, the HTTP endpoints	Jenkins, GitHub Actions
Smoke test	The deployed container is healthy, is the expected build, serves data and is being scraped	Jenkins (after every deploy), GitHub Actions

7. Results

7.1 Pipeline runs

Build	Trigger	What we did	Result
#1	Polling (first commit)	Normal run	Success in 54 s, build 1 deployed
#2	Manual	Normal run	Success in 22 s (layers cached), build 2 deployed
#3	Manual, SIMULATE_BAD_DEPLOY=true	Deployed a build whose /health returns 503	Smoke test failed; build 2 automatically restored and verified; PipelineFailed alert sent
#4	Polling, after a push	Normal run	Success, deployed automatically within a minute of the push
#5	Polling, after a push	Commit with a deliberately failing test	Stopped at Build & Test (1 failed, 22 passed); the next four stages were skipped; build 4 stayed live; PipelineFailed alert sent
#6	Polling, after a push	Reverted the bad test	Success, build 6 deployed; PipelineFailed resolved ("Pipeline is green again")

Jenkins / Sensor Dashboard CI/CD / #5

Timestamps [View as plain text](#)

☒ System clock time
☒ Use browser timezone
☐ Elapsed time
☐ None

```

13:20:17 sensor_dashboard/wsgi.py          2      2      0%
13:20:17 -----
13:20:17 TOTAL                                240      21      91%
13:20:17 ===== short test summary info =====
13:20:17 FAILED tests/test_api.py::test_readings_api_returns_three_sensors - Assertion...
13:20:17 ===== 1 failed, 22 passed in 0.77s =====
Post stage
[Pipeline] sh
13:20:18 + mkdir -p reports
13:20:18 + docker cp sensor-tests-5:/reports/junit.xml reports/junit.xml
13:20:18 + docker rm -f sensor-tests-5
13:20:18 + docker rmi sensor-dashboard:test-5
[Pipeline] junit
13:20:18 Recording test results
13:20:18 [Checks API] No suitable checks publisher found.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Validate Monitoring)
Stage "Validate Monitoring" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build Image)
Stage "Build Image" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
Stage "Deploy" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
  
```

Build #5: the failing test blocked the deployment

Jenkins / Sensor Dashboard CI/CD / #3

Timestamps [View as plain text](#)

☒ System clock time
☒ Use browser timezone
☐ Elapsed time
☐ None

```

[Pipeline] stage
[Pipeline] { (Smoke Test)
[Pipeline] sh
13:15:33 + docker run --rm -i --network sensor-net -e EXPECTED_BUILD=3 sensor-dashboard:3 python -
13:15:34 Smoke testing http://dashboard:8000 (expected build: 3)
13:16:20 FAIL app answers /health with status ok: /health returned HTTP 503
Post stage
[Pipeline] sh
13:16:21 + sh scripts/rollback.sh
13:16:21 Smoke test failed. Rolling back to the last good build (sensor-dashboard:stable)...
13:16:21 Container sensor-dashboard Recreate
13:16:22 Container sensor-dashboard Recreated
13:16:22 Container sensor-dashboard Starting
13:16:22 Container sensor-dashboard Started
13:16:23 Smoke testing http://dashboard:8000 (expected build: any)
13:16:23 PASS app answers /health with status ok
13:16:23 PASS API returns readings for 3 sensors
13:16:23 PASS /metrics exposes sensor metrics
13:16:23 PASS Prometheus is scraping the new container
13:16:23 Smoke test passed
13:16:23 Rollback complete: the previous build is serving traffic again.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] sh
13:16:24 + docker images sensor-dashboard --format {{.Tag}}
13:16:24 + grep -E ^[0-9]+$
13:16:24 + sort -n
13:16:24 + head -n -5
13:16:24 + xargs -r -I{} docker rmi sensor-dashboard:{}
13:16:24 + docker image prune -f
[Pipeline] sh
13:16:24 + sh scripts/notify_alertmanager.sh firing
13:16:24 Reporting the failed build to Alertmanager at http://host.docker.internal:9093
13:16:24
  
```

Build #3: failed smoke test and automatic rollback

Jenkins

Sensor Dashboard CI/CD

#6

Timestamps

View as plain text

System clock time

Use browser timezone

Elapsed time

None

```

[Pipeline] stage
[Pipeline] { (Smoke Test)
[Pipeline] sh
13:31:31 + docker run --rm -i --network sensor-net -e EXPECTED_BUILD=6 sensor-dashboard:6 python -
13:31:31 Smoke testing http://dashboard:8000 (expected build: 6)
13:31:31 PASS app answers /health with status ok
13:31:31 PASS API returns readings for 3 sensors
13:31:31 PASS /metrics exposes sensor metrics
13:31:31 PASS Prometheus is scraping the new container
13:31:31 Smoke test passed
Post stage
[Pipeline] sh
13:31:32 + docker tag sensor-dashboard:6 sensor-dashboard:stable
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] sh
13:31:33 + docker+ images sensor-dashboard --format {{.Tag}}
13:31:33 grep -E ^[0-9]+$
13:31:33 + sort -n
13:31:33 + head -n -5
13:31:33 + xargs -r -I{} docker rmi sensor-dashboard:{}
13:31:33 + docker image prune -f
[Pipeline] echo
13:31:33 Build 6 (aef0aea) is live on http://localhost:8000
[Pipeline] sh
13:31:33 + sh scripts/notify_alertmanager.sh resolved
13:31:33 Telling Alertmanager the pipeline is healthy again
13:31:33
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // timeout
[Pipeline] }
[Pipeline] // timestamps

```

Build #6: smoke test passing after a deploy

Jenkins

Sensor Dashboard CI/CD

#5

Tests

Status

Changes

Console Output

Delete build '#5'

Polling Log

Parameters

Git Build Data

Tests

Pipeline Overview

Edit build information

Restart from Stage

Replay

Pipeline Steps

Workspaces

Previous Build

Tests 23

1

22

1

Took 0.77 sec

All Failed Tests

Name	Age	Duration
test_readings_api_returns_three_sensors tests.test_api Regression	1	20 ms

All Tests

Package	Failed	Skipped	Passed	Total	Duration
tests	1 +1	0	22 -1	23	0.23 sec

REST API Jenkins 2.580.1

Jenkins test report for build #5

GitHub Actions independently passed or failed the same commits:

Platform
Solutions
Resources
Open Source
Enterprise
Pricing

Search
Sign in
Sign up

Yashvi2874 / DevOps_failure_monitoring
Public
Notifications
Fork 0
Star 0

Code
Issues
Pull requests
Actions
Projects
Security and quality
Insights

Actions
All workflows
CI
Management
Caches

All workflows
Showing runs from all workflows
Filter workflow runs

4 workflow runs
Workflow
Event
Status
Branch
Actor

✓
Revert the deliberately failing test
CI #4: Commit aef0aea pushed by Yashvi2874
main
2 minutes ago
40s

✗
Demo: push a failing test to show the pipeline blocks it
CI #3: Commit 228122f pushed by Yashvi2874
main
13 minutes ago
32s

✓
Merge branch 'fix/pipeline-alerts'
CI #2: Commit 0fa0ee6 pushed by Yashvi2874
main
15 minutes ago
52s

✓
Merge branch 'feature/github-actions'
CI #1: Commit d4e9057 pushed by Yashvi2874
main
20 minutes ago
56s

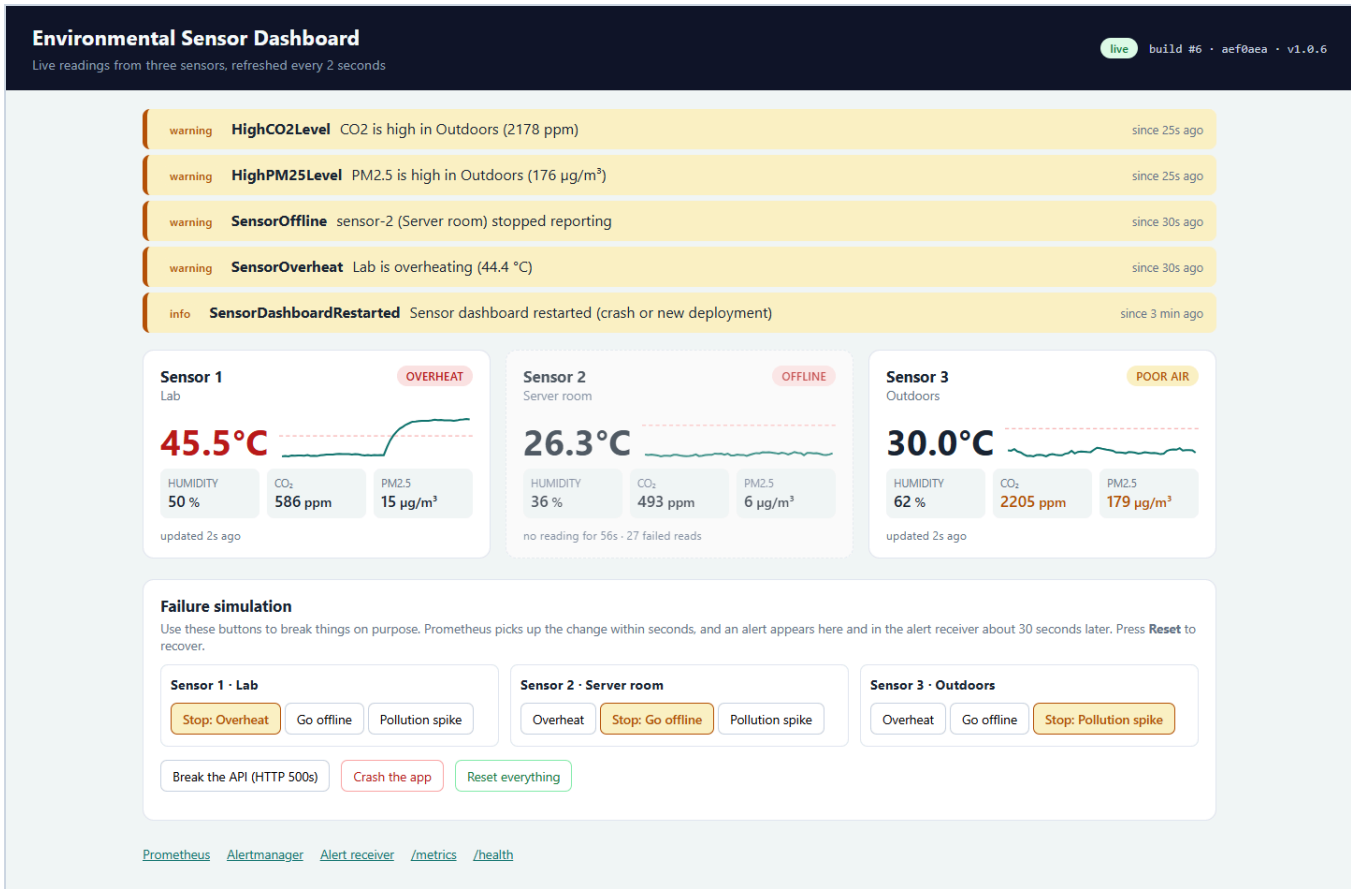
GitHub Actions runs

7.2 Failure detection

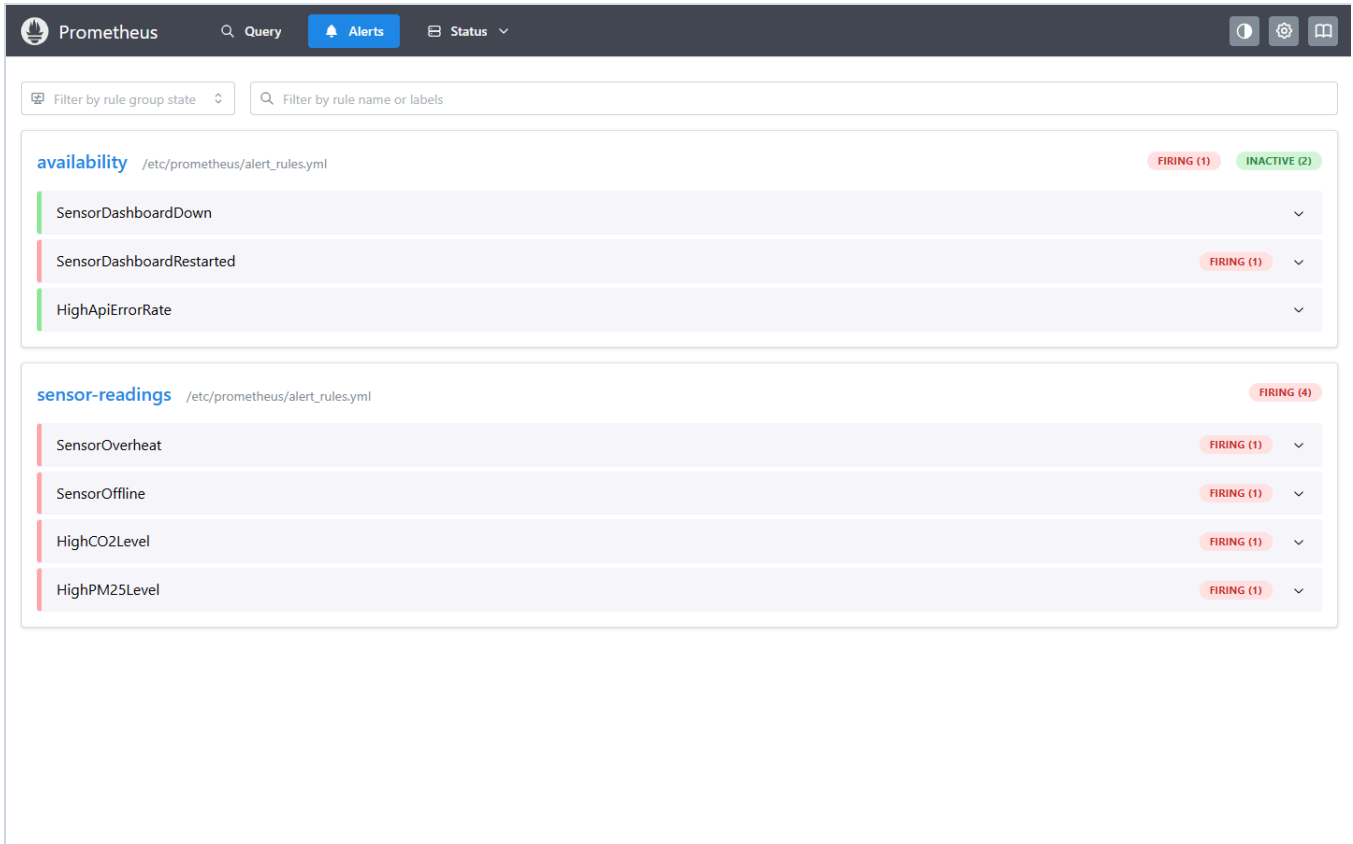
Times are measured from triggering the failure to the notification appearing in the alert receiver.

Scenario	Alert(s)	Channel	Time to notify
Sensor 1 overheats	SensorOverheat	team-chat	30–41 s
Sensor 2 goes offline	SensorOffline	team-chat	30 s
Pollution spike on Sensor 3	HighCO2Level + HighPM25Level (one notification)	team-chat	~45 s
API returns HTTP 500	HighApiErrorRate	on-call-pager	53 s
App process crashes	Docker restarts it; SensorDashboardRestarted	team-chat	10 s
Container stopped	SensorDashboardDown (restart alert inhibited)	on-call-pager	44–48 s
Container started again	SensorDashboardDown resolved	on-call-pager	24–29 s

Each delay is the sum of the scrape interval (5 s), the rule’s `for` duration (10 to 30 s), the evaluation interval (5 s) and Alertmanager’s `group_wait` (5 s). Shorter durations would notify faster but would also fire on harmless blips; these values suit a demo while still filtering one-off spikes.



Dashboard during an incident: three sensors misbehaving, alerts shown as a banner



Prometheus alert rules while the incident is active

Alertmanager

Alerts

Silences

Status

Settings

Documentation

New Silence

Filter

Group

Receiver: All

☐ Silenced

☐ Inhibited

☐ Muted

+

Silence

Custom matcher, e.g. `env="production"`

+ Expand all groups

+ team-chat

job="sensor-dashboard"

+

1 alert

+ team-chat

job="sensor-dashboard"

+

sensor="sensor-1"

+

1 alert

+ team-chat

job="sensor-dashboard"

+

sensor="sensor-2"

+

1 alert

+ team-chat

job="sensor-dashboard"

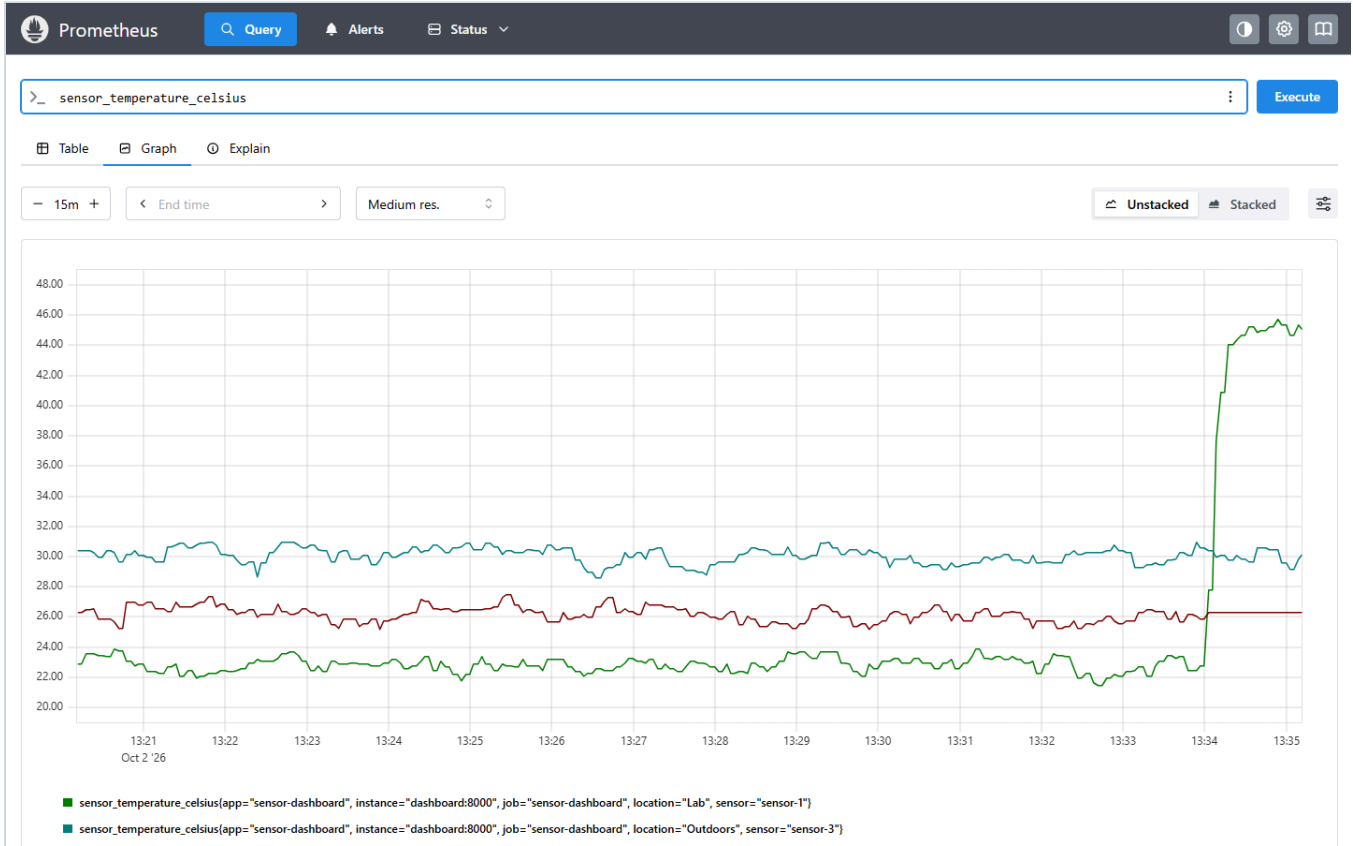
+

sensor="sensor-3"

+

2 alerts

Alertmanager grouping alerts per sensor (Sensor 3 has two)



Temperature history in Prometheus: Sensor 1 overheating

Alert receiver

Notifications delivered by Alertmanager (newest first). This page refreshes every 5 seconds.

8 firing 8 resolved

Received (UTC)	Channel	Status	Alert	Severity	Sensor	Summary
2026-10-02T13:37:50+00:00	on-call-pager	RESOLVED	SensorDashboardDown	critical	-	Sensor dashboard is unreachable
2026-10-02T13:37:40+00:00	team-chat	FIRING	SensorDashboardRestarted	info	-	Sensor dashboard restarted (crash or new deployment)
2026-10-02T13:37:20+00:00	on-call-pager	FIRING	SensorDashboardDown	critical	-	Sensor dashboard is unreachable
2026-10-02T13:36:40+00:00	team-chat	RESOLVED	SensorDashboardRestarted	info	-	Sensor dashboard restarted (crash or new deployment)
2026-10-02T13:36:36+00:00	team-chat	RESOLVED	HighPM25Level	warning	sensor-3	PM2.5 is high in Outdoors (100 µg/m³)
2026-10-02T13:36:31+00:00	team-chat	RESOLVED	SensorOverheat	warning	sensor-1	Lab is overheating (36.1 °C)
2026-10-02T13:36:06+00:00	team-chat	RESOLVED	HighCO2Level	warning	sensor-3	CO2 is high in Outdoors (1813 ppm)
2026-10-02T13:36:06+00:00	team-chat	FIRING	HighPM25Level	warning	sensor-3	PM2.5 is high in Outdoors (176 µg/m³)
2026-10-02T13:36:01+00:00	team-chat	RESOLVED	SensorOffline	warning	sensor-2	sensor-2 (Server room) stopped reporting
2026-10-02T13:34:36+00:00	team-chat	FIRING	HighCO2Level	warning	sensor-3	CO2 is high in Outdoors (2178 ppm)
2026-10-02T13:34:36+00:00	team-chat	FIRING	HighPM25Level	warning	sensor-3	PM2.5 is high in Outdoors (176 µg/m³)
2026-10-02T13:34:31+00:00	team-chat	FIRING	SensorOffline	warning	sensor-2	sensor-2 (Server room) stopped reporting
2026-10-02T13:34:31+00:00	team-chat	FIRING	SensorOverheat	warning	sensor-1	Lab is overheating (44.4 °C)
2026-10-02T13:31:54+00:00	team-chat	RESOLVED	PipelineFailed	warning	-	Pipeline is green again (build #6 passed)
2026-10-02T13:31:40+00:00	team-chat	FIRING	SensorDashboardRestarted	info	-	Sensor dashboard restarted (crash or new deployment)
2026-10-02T13:23:55+00:00	team-chat	RESOLVED	SensorDashboardRestarted	info	-	Sensor dashboard restarted (crash or new deployment)

Alert receiver: notifications on both channels, firing and resolved

Environmental Sensor Dashboard

Live readings from three sensors, refreshed every 2 seconds

live build #6 · aef0aea · v1.0.6

No active alerts. Everything looks healthy.

Sensor 1

Lab

ONLINE

22.7°C

HUMIDITY
43 %

CO₂
647 ppm

PM2.5
15 µg/m³

updated 1s ago

Sensor 2

Server room

ONLINE

26.4°C

HUMIDITY
35 %

CO₂
502 ppm

PM2.5
8 µg/m³

updated 1s ago

Sensor 3

Outdoors

ONLINE

30.1°C

HUMIDITY
61 %

CO₂
432 ppm

PM2.5
46 µg/m³

updated 1s ago

Failure simulation

Use these buttons to break things on purpose. Prometheus picks up the change within seconds, and an alert appears here and in the alert receiver about 30 seconds later. Press **Reset** to recover.

Sensor 1 · Lab

Overheat

Go offline

Pollution spike

Sensor 2 · Server room

Overheat

Go offline

Pollution spike

Sensor 3 · Outdoors

Overheat

Go offline

Pollution spike

Break the API (HTTP 500s)

Crash the app

Reset everything

[Prometheus](#) [Alertmanager](#) [Alert receiver](#) [/metrics](#) [/health](#)

The dashboard in its normal state

8. Challenges and how we solved them

1. **Deploying from inside a Jenkins container.** Our first idea was to mount `prometheus.yml` into the Prometheus container from the project folder. That fails when Jenkins runs the deployment: Jenkins's workspace is inside the Jenkins container, and the host's Docker engine can't see that path. We copied the config files into small custom images instead, which also puts config changes through the same pipeline as code.
2. **Docker socket permissions.** The socket inside the Jenkins container belongs to the root group, so the `jenkins` user couldn't use it. We added that group to the container (`group_add`) instead of running Jenkins as root. The group id is a setting, because it differs on Linux hosts.
3. **Alertmanager remembers what it sent.** When we restarted Alertmanager in the middle of an active alert, it treated the same alert as already delivered and stayed silent. We now redeploy Alertmanager only when its configuration changes, and noted this in the troubleshooting guide.
4. **Everything restarted on the first Jenkins deploy.** The stack had first been started from our own working copy. The images Jenkins built from its checkout were not byte-for-byte the same (different line endings and file timestamps), so Compose replaced every container once. From then on only services whose image really changed are replaced. We also added a `.gitattributes` file so shell scripts and configs always keep Unix (LF) line endings, because a script with Windows line endings fails on Linux.
5. **Port clash.** Port 8080 was already used by another program on our machine, so the Jenkins port is configurable (`JENKINS_PORT`).
6. **Tests could not import the app inside the container.** `pytest` started from the container's working directory did not add it to the import path. Setting `pythonpath = .` in `setup.cfg` fixed it for every environment.

9. Conclusion

The project covers the whole DevOps loop: code → build → test → deploy → monitor → alert. Every change goes through version control and an automated pipeline that refuses to deploy untested code and undoes a bad release on its own. Docker makes the app run the same way on every machine, from a developer's laptop to GitHub's test machines. Prometheus and Alertmanager turn failures into clear, grouped notifications within about half a minute, and confirm when the problem is fixed. Each of these behaviours was demonstrated with a real failure, not only described.

10. Future scope

- Replace the simulator with real sensors (for example an ESP32 publishing over MQTT) without changing the pipeline or the monitoring.
- Add Grafana dashboards on top of Prometheus for long-term trends.
- Send notifications to Slack, email or Telegram by editing the Alertmanager receivers.
- Deploy to a cloud server (for example Linode) over SSH, pushing images to a private registry such as Nexus instead of building on the target machine.
- Use a GitHub webhook (through a public Jenkins URL) instead of polling.
- Run Jenkins builds on separate agents instead of on the controller, and move to Kubernetes for zero-downtime rolling updates.

11. References

1. Jenkins documentation: Pipeline syntax and Configuration as Code. <https://www.jenkins.io/doc/>
2. Prometheus documentation: configuration, alerting rules and unit testing rules. <https://prometheus.io/docs/>
3. Alertmanager documentation: routing, grouping and inhibition. <https://prometheus.io/docs/alerting/latest/alertmanager/>
4. Docker documentation: multi-stage builds and Compose. <https://docs.docker.com/>
5. prometheus_client for Python. https://github.com/prometheus/client_python
6. Flask documentation. <https://flask.palletsprojects.com/>